

Now, let us try to understand the program. Line 1 imports the *random* module of NumPy. Lines 2 and 3 import libraries Matplotlib and seaborn for plotting. Line 5 generates a binomial distribution with parameters *n* (number of trials) and *p* (probability of success). Binomial distribution is a discrete probability distribution, which gives the number of successes in a sequence of *n* independent experiments. The third parameter called *size* decides the shape of the output. Line 6 plots the histogram of the data generated. Notice that a KDE plot is also plotted because the parameter *kde* is set as true. The third parameter *colour* decides the colour of the plot, which in this case is red ('r'). Line 7 generates a Poisson distribution (named after the French mathematician Siméon Denis Poisson). The Poisson distribution is a discrete probability distribution which gives the limit of the binomial distribution. The parameter *lam* sets the expected number of events occurring in a fixed-time interval. Here also the parameter *size* decides the shape of the output. Line 8 plots the data generated as a histogram in green. Line 9 generates an exponential distribution with a size of 1000. Line 10 plots the data generated as a histogram in blue. Finally, Line 11 displays all the plots generated. Figure 5 shows the three probability distributions plotted on execution of the program. The *random* module of NumPy offers a large number of other probability distributions like Dirichlet distribution, Gamma distribution, geometric distribution, Laplace distribution, etc. It would be really rewarding if you could work and get familiarised with more of them.

Now, let us learn about linear regression. Linear regression analysis can be used to predict the value of a variable based on the value of another variable. One of the important applications of linear regression is

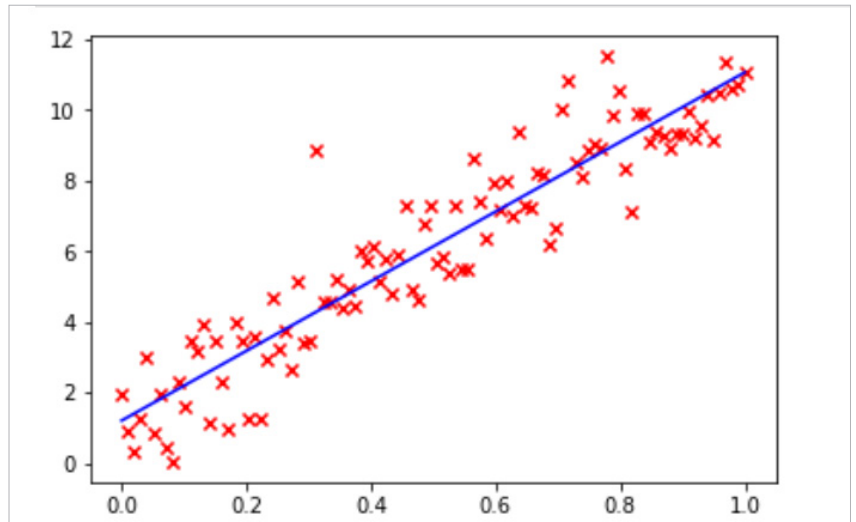


Figure 7: Linear regression scatter plot 1

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy import stats
4 a = 10
5 b = 1
6 x = np.linspace(0,1,100)
7 y = a*x+b+np.random.randn(len(x))
8 plt.scatter(x, y, color = "y", marker = "o")
9 m, c, r, p, std_err = stats.linregress(x, y)
10 y_l = list(m*x+c)
11 plt.plot(x,y_l,color = "g",)
12 plt.show()

```

Figure 8: Linear regression with SciPy

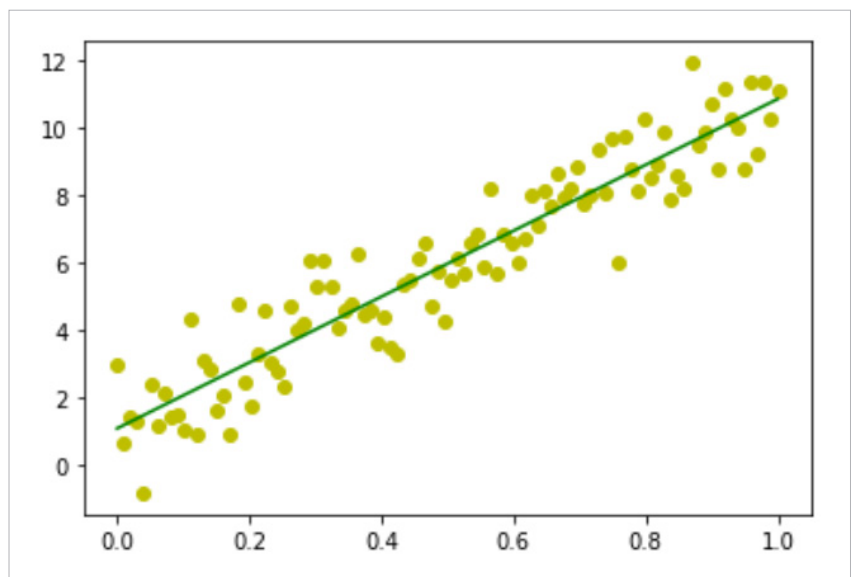


Figure 9: Linear regression scatter plot 2